MatchFusion SRS — Member 1: Md. Mehedi Hassan; ID: 24524203046

Introduction | Prepared from the provided CSE 3206 SRS outline

1. Introduction

1.1 Purpose

MatchFusion is a web-based sports community and match analysis platform for football and cricket. It brings live, upcoming and completed match information, team and player directories, AI-assisted match insights, and a registered-user community (posts, comments, reactions and polls) into one application. This SRS defines the requirements that will guide the development and testing of the MatchFusion system.

1.2 Intended Audience

- Sports fans (visitors) who want to follow matches, scores, fixtures, teams and players in one place.
- Registered users who want to take part in discussions, comment on posts, react and vote in polls.
- Course supervisors and examiners who will review the system against its requirements.
- Developers and testers who will use the SRS as a reference during development, testing and handover.

1.3 Intended Use

The SRS will be used as the common requirements reference for the four-member development team. It describes the expected behavior, users, scope, constraints, assumptions and quality requirements of MatchFusion. The requirements are written against the existing React, Express and Supabase PostgreSQL implementation, so that each requirement can be traced to a module and verified by a test.

1.4 Product Scope

MatchFusion will allow visitors to browse, filter and search football and cricket matches, view match details, explore team and player directories, and read AI-generated match insights. Registered users will additionally be able to create posts, comment, react to posts, create polls and vote. Administrators of the platform (system maintainers) will manage configuration, API credentials, database migrations and health monitoring outside the application interface.

The system collects match data from API-Football (football) and CricketData.org (cricket), normalizes it into a common match structure, caches it in memory, and stores match snapshots in a PostgreSQL database through Prisma ORM. AI insights combine a rule-based probability engine with a natural-language explanation generated by Google Gemini.

Out of scope for the current version:

- Sports other than football and cricket.
- WebSocket-based real-time push (the system uses scheduled refresh, caching and frontend polling).
- Email verification, password reset, social login, and an in-app administrator or moderation role.
- Trained and statistically calibrated machine-learning prediction models.
- Complete worldwide historical team and player coverage, and a native mobile application.

1.5 Risk Definitions

- External provider outage, rate limiting or quota exhaustion may make match data unavailable or stale.
- Differences between provider data formats may cause incorrect normalization or missing events and statistics.
- AI probabilities are rule-based heuristic estimates and may be misread as validated predictions.
- Gemini service failure or a malformed AI response may prevent the natural-language explanation from being produced.
- Invalid input or repeated actions (for example duplicate votes) may corrupt community data.
- Unauthorized access or leaked secrets (database credentials, JWT secret, API keys) may expose user and system information.
- Free-tier hosting may put the backend to sleep, delaying the first response and stopping scheduled refresh.
- Temporary database connectivity failure may affect login, community features and match persistence.

2. Overall Description

2.1 User Classes and Characteristics

Visitor

A visitor has not logged in. A visitor can browse, filter and search matches, view match details and AI insights, explore teams and players, and read community posts and polls.

Registered User

A registered user has an account and is logged in. In addition to all visitor features, a registered user can view a profile with activity counts, manage posts, comment, react, create polls and vote.

System Maintainer

A system maintainer is a developer or operator who configures environment variables and API credentials, applies database migrations, deploys the system, runs tests and checks the health endpoints. This is not an in-app role (the user model has no role field).

2.2 User Needs

- Visitors need one place to follow live, upcoming and completed football and cricket matches.
- Visitors need quick search (including short names such as BD, AUS or EPL) with important competitions listed first.
- Sports enthusiasts need team and player information and, where available, match events and statistics.
- Data-driven fans need an understandable probability estimate with a short explanation.
- Community members need a simple way to post, comment, react and vote in polls linked to matches.
- System maintainers need health checks, controlled data refresh and clear configuration for deployment.

2.3 Operating Environment

- Web-based application accessible through a modern desktop or mobile web browser; an internet connection is required.
- Frontend: React 19 with Vite, Tailwind CSS, React Router and Axios.
- Backend: Node.js (version 18 or newer) with Express.js REST API, Prisma ORM and Supabase-hosted PostgreSQL.
- External services: API-Football (football data), CricketData.org (cricket data) and Google Gemini API (AI explanation).
- Planned hosting: Vercel for the frontend, Render for the backend and Supabase for the database.

2.4 Constraints

- The project is developed by a four-member student team within the available academic time.
- External sports APIs have free-tier quotas, so the backend must limit calls through caching, scheduled refresh and refresh cooldowns.
- Only football and cricket are supported; each sport has a different data structure and needs its own provider logic.
- Match events and statistics depend on the provider plan and may not exist for every match; AI probabilities are rule-based heuristics, not a trained model.
- Free hosting may sleep after inactivity, and the in-process scheduler stops while the backend sleeps.
- Private secrets must stay in backend environment variables, because frontend VITE_ variables can be exposed in the browser.

2.5 Assumptions

- Users have access to the internet and a supported web browser.
- Users provide valid information during registration and keep their credentials private.
- The external sports APIs are available, the API keys are valid and usage stays within quota; MatchFusion displays the data the providers return.
- The server and database are available during normal operation.

- A Gemini API key is configured; otherwise the system returns the mathematical estimate without the AI explanation.
- The frontend API base URL and the backend allowed origins are configured correctly for each deployment environment.

MatchFusion SRS — Member 3: Tabassum Binte Tariq Rabsa; ID: 24524203050

Functional Requirements | Prepared from the provided CSE 3206 SRS outline

3. Specific Requirements

3.1 Functional Requirements

The functional requirements below are an initial team draft. Each requirement uses the User Story and Confirmation format required by the lab manual. The team should review, renumber, remove, or add requirements after the final MatchFusion scope is agreed.

Account & Access

FR-01 — User Registration

User Story: As a new user, I want to create an account so that I can take part in the MatchFusion community.

Confirmation: The system shall validate that the name has at least 2 characters, the email is valid and not already used, the password has at least 6 characters and the confirmation password matches, and shall then create the account with a bcrypt-hashed password.

FR-02 — User Login

User Story: As a registered user, I want to log in so that I can access my account and protected features.

Confirmation: The system shall verify the email and password against the stored hash and return a signed JWT valid for 7 days together with the user information.

FR-03 — Session Restoration

User Story: As a logged-in user, I want my session restored when I refresh the website so that I do not have to log in again.

Confirmation: The system shall validate the stored token through the session-check endpoint on page load, restore the user state when the token is valid and clear the token when the server returns an authorization error.

FR-04 — Profile and Activity

User Story: As a registered user, I want to view my profile so that I can see my account details and activity.

Confirmation: The system shall display the name, email, account creation information, counts of posts, comments, reactions and votes, and recent authored posts.

FR-05 — Logout

User Story: As a logged-in user, I want to log out so that my account is not accessible on a shared device.

Confirmation: The system shall clear the stored token and user information from the browser and acknowledge the logout request.

Match Tracking

FR-06 — Browse Matches

User Story: As a visitor, I want to browse football and cricket matches so that I can follow live, upcoming and completed matches.

Confirmation: The system shall display matches with teams, score, status, competition and kickoff time using the common MatchFusion match structure.

FR-07 — Filter Matches

User Story: As a visitor, I want to filter matches by sport and status so that I can narrow the list.

Confirmation: The system shall allow filtering by sport (all, football, cricket) and by status (all, live, upcoming, completed).

FR-08 — Search Matches with Aliases

User Story: As a visitor, I want to search with short names such as BD, AUS or EPL so that I can find matches quickly.

Confirmation: The system shall expand known aliases to their full names and return matches whose searchable fields contain the search term or its aliases.

FR-09 — Sort and Prioritize Competitions

User Story: As a visitor, I want important competitions listed first so that minor domestic matches do not fill the list.

Confirmation: The system shall assign a priority tier to known competitions and shall support sort options such as most important, live first, upcoming soonest and latest results.

FR-10 — View Match Details

User Story: As a visitor, I want to open a match so that I can see detailed information.

Confirmation: The system shall display Overview, Events, Statistics, AI Insights and Discussion tabs, and shall show only the events and statistics that the provider has returned.

FR-11 — Manual Refresh

User Story: As a visitor, I want to refresh match data so that I can get the latest scores.

Confirmation: The system shall refresh the selected sport on request, shall apply a cooldown of 2 minutes per sport, and shall return the existing cached data with a notification when the cooldown is active.

FR-12 — Data Freshness Indication

User Story: As a visitor, I want to know how recent the displayed data is so that I can judge its reliability.

Confirmation: The system shall display data freshness information and shall make it clear when cached data from an earlier snapshot is being shown.

Sports Data Integration

FR-13 — Fetch Provider Data

User Story: As the system, I want to fetch football and cricket data from the providers so that users see current match information.

Confirmation: The system shall retrieve live, today, recent and upcoming football fixtures from API-Football and current and upcoming cricket matches from CricketData.org, and shall refresh each sport independently so that one provider failure does not stop the other.

FR-14 — Normalize Match Data

User Story: As the system, I want provider responses converted to one common format so that the frontend can treat all matches the same way.

Confirmation: The system shall map provider status codes to LIVE, COMPLETED, UPCOMING, CANCELLED or POSTPONED and shall keep sport-specific details (events, statistics, runs, wickets, overs, run rates) in structured fields.

FR-15 — Scheduled Refresh and Caching

User Story: As the system, I want to refresh data on a schedule and cache it so that API quota is protected.

Confirmation: The system shall refresh football and cricket data at a configurable interval (default 30 minutes), keep the results in an in-memory cache and remove duplicate matches using the external match ID.

FR-16 — Persist Match Snapshots

User Story: As the system, I want matches saved to the database so that data survives a server restart.

Confirmation: The system shall upsert each normalized match using the unique provider and external ID pair, creating a new record when none exists and updating the existing record otherwise.

FR-17 — Provider Failure Fallback

User Story: As a visitor, I want to still see matches when a provider fails so that the website is not empty.

Confirmation: The system shall load the previously saved match snapshots from the database when a provider request fails and shall serve them as cached data.

Teams & Players

FR-18 — Team Directory

User Story: As a visitor, I want to browse teams so that I can discover football and cricket teams.

Confirmation: The system shall list teams with name, logo, country, competition and available match information, with sport filter, search and pagination, reading from the Team table first and deriving teams from match data when the table has no records.

FR-19 — Team Details

User Story: As a visitor, I want to open a team so that I can see its squad and recent matches.

Confirmation: The system shall display the team information, available player squad, recent matches and available results and form.

FR-20 — Player Directory

User Story: As a visitor, I want to browse players so that I can find player information.

Confirmation: The system shall list players with name, sport, position, role, country, team and image where available, with server-side pagination (default 50 per page), reading from the Player table first and falling back to players found in match events.

FR-21 — Football Squad Synchronization

User Story: As a system maintainer, I want to import football squads so that the player directory has real records.

Confirmation: The system shall fetch squads from API-Football for known football teams (initially at most 20), create or update players by provider ID, link each player to its team and apply a one-hour cooldown between synchronizations.

AI Match Insights

FR-22 — Calculate Match Probabilities

User Story: As a user, I want a probability estimate for a match so that I can understand who is more likely to win.

Confirmation: The system shall calculate home, draw and away probabilities for football (summing to 100%) and home and away probabilities for cricket using the match status, score, elapsed time and run rates, and shall show the final result as 100% for completed matches.

FR-23 — Generate AI Explanation

User Story: As a user, I want a short written explanation so that I can understand the factors behind the estimate.

Confirmation: The system shall send the match data and the already calculated probabilities to Google Gemini, shall return key factors, a short explanation, a confidence description and a disclaimer, and shall not allow the explanation to change the calculated percentages.

FR-24 — Cache and Store AI Insights

User Story: As the system, I want to reuse recent insights so that AI calls and delays are reduced.

Confirmation: The system shall reuse a saved insight until it expires (15 minutes for live, 2 hours for upcoming and 7 days for completed matches) and shall store generated insights in the AIInsight table when the database record is available.

FR-25 — AI Fallback

User Story: As a user, I want to still see the numeric estimate when the AI service is unavailable.

Confirmation: The system shall return the mathematically calculated probabilities with a clear note when the Gemini key is missing or the AI request fails or returns a malformed response.

Community

FR-26 — Browse Community Feed

User Story: As a visitor, I want to read posts and polls so that I can follow fan discussions.

Confirmation: The system shall display posts and polls to all users, with categories for all discussions, live matches, upcoming predictions, completed debates and active polls.

FR-27 — Create Post

User Story: As a registered user, I want to create a post so that I can share my opinion about a match.

Confirmation: The system shall accept a post of up to 2000 characters with an optional match tag and status, and shall set the author from the verified JWT rather than from the request body.

FR-28 — Edit and Delete Post

User Story: As a registered user, I want to edit or delete my post so that I can correct or remove it.

Confirmation: The system shall allow an authenticated user to edit or delete a post that the user authored.

FR-29 — Comment on Post

User Story: As a registered user, I want to comment on a post so that I can join the discussion.

Confirmation: The system shall save a comment linked to the post and the authenticated user, shall display the comments of a post to all users and shall allow only the comment author to delete a comment.

FR-30 — React to Post

User Story: As a registered user, I want to like a post so that I can show support.

Confirmation: The system shall toggle the reaction, creating it if it does not exist and removing it if it does, so that one user cannot have duplicate reactions of the same type on a post.

FR-31 — Create Poll

User Story: As a registered user, I want to create a poll so that fans can vote on a match question.

Confirmation: The system shall require a non-empty question and between 2 and 6 options (the interface offers 2 to 4), and shall save the poll, its options and the optional match ID and status.

FR-32 — Vote in Poll

User Story: As a registered user, I want to vote in a poll so that my opinion is counted.

Confirmation: The system shall verify the user, reject a second vote by the same user on the same poll, verify that the option belongs to the poll, save the vote inside a database transaction and return the updated poll.

FR-33 — View Poll Results

User Story: As a user, I want to see poll results so that I can compare opinions.

Confirmation: The system shall calculate each option percentage from the actual vote counts and shall show 0% without error when a poll has no votes.

System Monitoring

FR-34 — Health and Platform Statistics

User Story: As a system maintainer, I want to check the system status so that I can detect configuration and database problems.

Confirmation: The system shall provide health, database connectivity and platform statistics endpoints that report the backend status, configuration status and database connection result.

Team note: The team must decide whether an in-app administrator or moderation role, notifications, WebSocket real-time updates, password reset and email verification will be added, and whether poll-to-match linking will be enforced with a verified match selector and a database relationship.

Responsibility

- Maintain the functional requirement IDs and ensure every major feature has a requirement.
- Trace requirements to development tasks and tests.
- Main development area: sports data integration, match center and search, and AI match insights.

3.2 Non-Functional Requirements

The following are an initial draft of quality requirements for MatchFusion.

NFR-01 — Performance

The system should serve match lists and team and player directories from cache or database instead of calling external providers on every page view, and should display normal pages and search results within an acceptable response time under normal expected load.

NFR-02 — Security

The system shall hash passwords with bcrypt, protect all write operations with JWT verification, take the acting user from the verified token and not from the request body, and send standard security headers such as X-Content-Type-Options and X-Frame-Options.

NFR-03 — Secret Management

Database credentials, the JWT secret and provider API keys shall be stored only in backend environment variables, shall not be placed in the frontend bundle and shall not be committed to a public repository.

NFR-04 — Usability

The system should provide a clear and consistent responsive interface with reusable components so that visitors and registered users can find matches, open details and use community features without unnecessary complexity.

NFR-05 — Reliability

The system should handle provider and database failures without losing existing data, should refresh each sport independently and should fall back to saved snapshots instead of showing an empty website.

NFR-06 — Availability

The system should be available during the expected operating period, except for planned maintenance or unavoidable interruptions such as free-tier hosting sleep, which may delay the first response.

NFR-07 — Maintainability

The software should be organized into understandable modules (routes, services, middleware, library functions, pages and components) so that a feature can be modified or extended with minimal impact on unrelated components.

NFR-08 — Scalability

The system should be designed so that the number of users, matches, teams, players, posts and polls can increase without a complete redesign, and so that another sport can be added through a new provider module.

NFR-09 — Data Integrity

The system shall use database constraints to prevent duplicate users by email, duplicate matches by provider and external ID, duplicate teams by sport and name, duplicate reactions and duplicate votes by the same user on a poll.

NFR-10 — Data Accuracy and Transparency

AI results shall be labeled as estimates with the calculation method, confidence and a disclaimer; the system shall not invent statistics, shall show missing events or statistics as unavailable and shall indicate when cached data is displayed.

NFR-11 — API Quota Efficiency

The system should reduce external API usage through caching, a configurable scheduled refresh interval, a 2-minute manual refresh cooldown per sport and a one-hour squad synchronization cooldown.

NFR-12 — Compatibility

The web application should work correctly on commonly used modern web browsers and on supported desktop and mobile screen sizes defined by the team.

NFR-13 — Privacy

The system shall never return password hashes or credentials in an API response and should limit personal account information to the account owner.

NFR-14 — Error Handling

The system should provide understandable error messages, return correct HTTP status codes for validation and authorization errors and should not expose sensitive technical information to normal users.

NFR-15 — Backup and Recovery

The team should use the database backup features of the hosting platform, apply schema changes only through Prisma migrations and should not run destructive reset commands on the production database.

NFR-16 — Testability

The backend should include automated tests runnable with a single command for authentication, normalization, search, polls and AI probability logic, and database-dependent integration tests should be runnable separately.

NFR-17 — Accessibility

The interface should use readable text, understandable labels, adequate contrast on the dark theme and keyboard-friendly controls where practical.